

DATA MODELING MASTERY SERIES

Star Schema, Snowflake Schema, SCD & Modeling for Pipelines

Book 2 of 4 — Data Modeling Mastery Series

For Data Engineers building production-grade, practical data modeling skills

Table of Contents

How To Use This Book	3
PART 5 — STAR SCHEMA	4
5.1 What is a Star Schema?	4
5.2 Structure, Advantages, Disadvantages	4
5.3 Star Schema vs Normalized Model	4
5.4 Star Schema vs Snowflake Schema (preview)	5
5.5 Worked Example 1 — E-commerce	5
5.6 Worked Example 2 — Banking Transactions	7
5.7 Worked Example 3 — Financial Reporting	8
5.8 Worked Example 4 — Subscription Business	9
PART 6 — SNOWFLAKE SCHEMA	11
6.1 What is a Snowflake Schema?	11
6.2 How It Differs From Star Schema	11
6.3 Normalized Dimensions	11
6.4 Advantages	12
6.5 Disadvantages	12
6.6 Performance Considerations	12
6.7 Storage Considerations	12
6.8 When to Use Star Schema	12
6.9 When to Use Snowflake Schema	13
PART 7 — SLOWLY CHANGING DIMENSIONS (DEEP DIVE)	14
7.1 What is SCD and Why Historical Data Matters	14
7.2 SCD Type 0 — Retain Original	14
7.3 SCD Type 1 — Overwrite	15
7.4 SCD Type 3 — Add a Column for Previous Value	15
7.5 SCD Type 2 — Add a New Row (Full History) — VERY DEEP DIVE	17
7.6 SCD Type 4 — History Table	21
7.7 SCD Type 6 — Hybrid (1 + 2 + 3)	21
7.8 SCD Type Comparison Summary	23
PART 8 — ADVANCED DATA MODELING CONCEPTS	24
8.1 Hierarchies, Parent-Child, and Recursive Relationships	24
8.2 Many-to-Many Relationships & Bridge Tables	25
8.3 Late-Arriving Dimensions and Facts	25
8.4 Unknown Members	26
8.5 Multi-Valued Attributes	27
8.6 Hierarchical Dimensions (Summary)	27
PART 9 — DATA MODELING FOR ETL/ELT	28
9.1 Source System Analysis & Source-to-Target Mapping	28
9.2 Data Profiling, Identifying Business Keys, Identifying Grain	28
9.3 Full Load vs Incremental Load	29
9.4 CDC (Change Data Capture)	29
9.5 MERGE / UPSERT	30
9.6 Watermarking	30
9.7 Deduplication	31
9.8 Data Reconciliation	31
9.9 Schema Evolution	32
9.10 Late-Arriving Data and Backfills	32

9.11 Idempotency	33
9.12 Designing a Model When Source Data is Messy	33
Glossary — Book 2 Terms	34
Book 2 Practical Scenarios (15 Scenarios)	36

How To Use This Book

This is **Book 2 of 4**. Book 1 covered fundamentals, database theory, and fact/dimension basics. This book goes deep on the **Star Schema** (with four full worked industry examples), the **Snowflake Schema**, a **full deep dive on Slowly Changing Dimensions**, advanced modeling concepts, and how modeling connects to **ETL/ELT pipeline design**.

Same teaching pattern as Book 1:

CONCEPT -> What/Why -> real-world example -> table example -> diagram
-> SQL -> when to use/not use -> pros/cons -> common mistakes
-> practical scenario

Series map:

Book	Covers
1	Fundamentals, database theory, fact/dimension basics
2 (this book)	Star & Snowflake schemas, SCD deep dive, modeling for ETL/ELT
3	Medallion architecture, Snowflake & dbt implementation, real-world project
4	Practical scenarios, system design, practice exercises

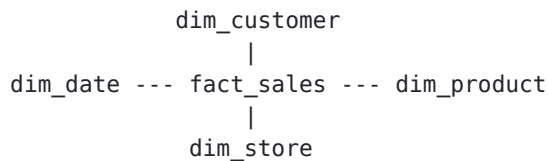
PART 5 — STAR SCHEMA

5.1 What is a Star Schema?

What is it? A dimensional model where a central **fact table** connects directly to a set of **denormalized dimension tables**, each one join away — the diagram resembles a star, with the fact table at the center and dimensions radiating outward.

Why it matters: It's the most common physical structure for BI-facing data marts because it maps directly onto how business users think ("show me X by Y by Z") and because it minimizes the number of joins a query needs, which both simplifies SQL and improves query performance.

Diagram:



5.2 Structure, Advantages, Disadvantages

Structure: One fact table (large, mostly numeric + FKs) + N dimension tables (smaller, denormalized, descriptive), with the fact table's foreign keys pointing directly at each dimension's surrogate key — no intermediate lookup tables.

Advantages: - Minimal joins (one hop from fact to any dimension attribute) - Intuitive for business users and BI tools (Tableau, Power BI, Looker all model semantic layers around star schemas) - Query engines optimize well for this shape (small dimension filtered first, used to prune the large fact table)

Disadvantages: - Denormalized dimensions carry redundant data (e.g., category_name repeated on every product row in that category) - Updating a shared attribute across a denormalized dimension can touch many rows (mitigated in warehouses since dimensions are refreshed by pipelines, not hand-edited)

5.3 Star Schema vs Normalized Model

	Star Schema	Normalized (3NF) Model
Joins per query	Few (1 hop to any dimension)	Many (traverse multiple normalized tables)
Redundancy	Higher (denormalized dimensions)	Lower
Query simplicity	High	Low — requires understanding the full schema
Best for	Analytical/BI queries	Transactional writes, OLTP

Practical scenario: “Why is a heavily normalized schema a poor fit for a BI dashboard, even if it's ‘more correct’ from a database theory standpoint?” **Approach:** A normal-

ized schema minimizes redundancy, which is the right goal for transactional writes, but it requires many joins to answer a typical business question (“revenue by category by region”), which is slower and much harder for both BI tools and business users to construct correctly. A star schema trades some redundancy for query simplicity and speed, which is exactly the right trade-off for read-heavy analytical workloads.

5.4 Star Schema vs Snowflake Schema (preview)

A Snowflake Schema normalizes dimensions (e.g., splitting `dim_product` into `dim_product` -> `dim_category` -> `dim_department`), trading a few extra joins for less redundancy. Full treatment is in Part 6 — the short version: **default to Star Schema**; consider Snowflaking only a specific large, frequently-changing hierarchy where the redundancy cost is genuinely high.

5.5 Worked Example 1 — E-commerce

Grain: One row per product per order (one row per order line item).

Facts identified: `quantity` (additive), `unit_price` (non-additive), `line_amount` (additive), `discount_amount` (additive).

Dimensions identified: `dim_customer`, `dim_product`, `dim_store` (or `dim_channel` for online-only retailers), `dim_date`, plus degenerate dimension `order_number`.

Relationships: `fact_order_item` many-to-one to each dimension; `dim_date` is role-played as `order_date` (primary) and could be role-played again as `ship_date` if fulfillment is tracked in the same fact (usually better split into its own accumulating snapshot fact — see Book 1, section 4.6).

Table structures:

```
CREATE TABLE dim_customer (
  customer_sk      BIGINT PRIMARY KEY,
  customer_id      VARCHAR(50),      -- natural/business key
  full_name        VARCHAR(200),
  email            VARCHAR(255),
  signup_date      DATE,
  customer_segment VARCHAR(50),
  is_current       BOOLEAN,
  effective_start   DATE,
  effective_end     DATE
);

CREATE TABLE dim_product (
  product_sk      BIGINT PRIMARY KEY,
  product_id       VARCHAR(50),
  product_name     VARCHAR(200),
  category         VARCHAR(100),
  subcategory      VARCHAR(100),
  brand            VARCHAR(100),
  unit_cost        DECIMAL(10,2)
);

CREATE TABLE dim_date (
  date_sk          INT PRIMARY KEY,      -- yyyymmdd
  full_date        DATE,
  day_of_week      VARCHAR(10),
  month            INT,
  month_name       VARCHAR(20),
```

```
    quarter      INT,
    year         INT,
    is_weekend   BOOLEAN
);

CREATE TABLE fact_order_item (
    order_item_sk BIGINT PRIMARY KEY,
    order_number  VARCHAR(50), -- degenerate dimension
    customer_sk   BIGINT REFERENCES dim_customer(customer_sk),
    product_sk    BIGINT REFERENCES dim_product(product_sk),
    date_sk       INT REFERENCES dim_date(date_sk),
    quantity      INT,
    unit_price    DECIMAL(10,2),
    discount_amount DECIMAL(10,2),
    line_amount   DECIMAL(10,2)
);
```

Design decisions explained: - `dim_customer` is built for SCD Type 2 (effective dates + `is_current`) because customer segment reclassification should be historically accurate for past-period reporting. - `order_number` stays degenerate — there’s no useful attribute of “the order” beyond identifying it; order-level attributes like status belong in a separate accumulating snapshot fact if fulfillment tracking is needed. - `unit_price` is kept even though it’s non-additive, because it’s needed for line-level analysis (e.g., discount rate); aggregation should always go through `line_amount`, never `AVG(unit_price)`.

Example query:

```
SELECT d.year, d.month_name, p.category, SUM(f.line_amount) AS revenue
FROM fact_order_item f
JOIN dim_date d ON f.date_sk = d.date_sk
JOIN dim_product p ON f.product_sk = p.product_sk
GROUP BY d.year, d.month_name, p.category
ORDER BY d.year, d.month_name;
```

5.6 Worked Example 2 — Banking Transactions

Grain: One row per financial transaction posted to an account (deposit, withdrawal, transfer leg, fee).

Facts identified: `transaction_amount` (additive, signed +/- for debit/credit), `running_balance_after` (semi-additive — a snapshot value, not summable across time), `fee_amount` (additive).

Dimensions identified: `dim_account`, `dim_customer`, `dim_branch`, `dim_date`, `dim_transaction_type` (a junk-dimension-like small lookup of transaction type/channel combinations).

Relationships: many transactions per account per day; `dim_account` and `dim_customer` are separate because one customer can hold multiple accounts (joint accounts introduce a many-to-many relationship handled via a bridge table — see Part 8).

Table structures:

```
CREATE TABLE dim_account (
    account_sk      BIGINT PRIMARY KEY,
    account_id      VARCHAR(50),
    account_type    VARCHAR(30),      -- checking, savings, loan
    opened_date     DATE,
    branch_sk       BIGINT,
    is_current      BOOLEAN,
    effective_start DATE,
    effective_end   DATE
);

CREATE TABLE fact_transaction (
    transaction_sk  BIGINT PRIMARY KEY,
    transaction_id  VARCHAR(50),      -- degenerate dimension
    account_sk      BIGINT REFERENCES dim_account(account_sk),
    customer_sk     BIGINT REFERENCES dim_customer(customer_sk),
    date_sk         INT REFERENCES dim_date(date_sk),
    transaction_type_sk BIGINT,
    transaction_amount DECIMAL(18,2), -- signed
    fee_amount      DECIMAL(10,2)
);
```

Design decisions explained: Transaction fact tables in banking are almost always **immutable**, **append-only**, **transaction-grain** facts — a corrected/reversed transaction is modeled as a *new* offsetting transaction row, never an UPDATE, preserving a full audit trail (a regulatory requirement in most jurisdictions). Account *balances* are handled by a separate **periodic snapshot fact** (`fact_daily_account_balance`, one row per account per day) since balance is semi-additive and needs regular checkpoints independent of transaction activity.

Example query — average daily transaction volume by branch:

```
SELECT b.branch_name, d.year, d.month,
       COUNT(*) AS txn_count,
       SUM(f.transaction_amount) AS net_amount
FROM fact_transaction f
JOIN dim_account a ON f.account_sk = a.account_sk
JOIN dim_branch b ON a.branch_sk = b.branch_sk
JOIN dim_date d ON f.date_sk = d.date_sk
GROUP BY b.branch_name, d.year, d.month;
```

5.7 Worked Example 3 — Financial Reporting

Grain: One row per general ledger line item posted per accounting period (a classic financial reporting fact — “one row per GL line per period per account”).

Facts identified: debit_amount, credit_amount (both additive), local_currency_amount, reporting_currency_amount (additive after currency conversion is applied at load time).

Dimensions identified: dim_gl_account (chart of accounts, often hierarchical — a natural Snowflake Schema candidate, see Part 6), dim_cost_center, dim_period (a specialized dim_date variant at fiscal-period grain rather than daily grain), dim_currency, dim_entity (legal entity/subsidiary, for consolidated reporting).

Relationships: the chart of accounts hierarchy (account -> sub-category -> category -> financial statement line) is usually modeled as a **snowflaked** or **hierarchical/flattened** dimension (covered in Part 6 and 8) since financial reporting requires roll-ups (e.g., “Total Operating Expenses”) that must stay perfectly consistent with the official chart of accounts.

Table structures:

```
CREATE TABLE dim_gl_account (
    gl_account_sk      BIGINT PRIMARY KEY,
    gl_account_id      VARCHAR(30),
    account_name       VARCHAR(200),
    account_category   VARCHAR(100), -- flattened hierarchy level 1
    account_subcategory VARCHAR(100), -- flattened hierarchy level 2
    financial_statement_line VARCHAR(100) -- flattened hierarchy level 3
);

CREATE TABLE dim_period (
    period_sk      INT PRIMARY KEY, -- yyyyymm
    fiscal_year    INT,
    fiscal_quarter INT,
    fiscal_month   INT,
    period_end_date DATE
);

CREATE TABLE fact_gl_line (
    gl_line_sk      BIGINT PRIMARY KEY,
    journal_entry_id VARCHAR(50), -- degenerate dimension
    gl_account_sk    BIGINT REFERENCES dim_gl_account(gl_account_sk),
    cost_center_sk   BIGINT,
    entity_sk        BIGINT,
    period_sk        INT REFERENCES dim_period(period_sk),
    currency_sk      BIGINT,
    debit_amount     DECIMAL(18,2),
    credit_amount     DECIMAL(18,2),
    reporting_currency_amount DECIMAL(18,2)
);
```

Design decisions explained: Debit and credit are kept as **two separate additive columns** rather than one signed amount column — this matches how accountants think and validate (SUM(debit_amount) = SUM(credit_amount) per journal entry is a standard balance check), even though it could technically be modeled as a single signed measure.

Interview question: “Why would a financial reporting fact table keep separate debit_amount and credit_amount columns instead of one signed amount column?” **Strong answer:** Double-entry accounting fundamentally validates that debits equal credits per journal entry — keeping them as separate additive columns lets that validation and stan-

standard financial statement construction (which reports debits and credits differently depending on account type) work naturally in SQL, whereas a single signed column would require re-deriving the debit/credit split contextually.

5.8 Worked Example 4 — Subscription Business

Grain: For the primary billing fact — **one row per subscription per billing period** (e.g., one row per subscriber per month).

Facts identified: mrr_amount (Monthly Recurring Revenue, additive within a period), add_on_amount, discount_amount, days_active_in_period (semi-additive — meaningful for one subscriber-period, not simply summable across subscribers into something interpretable beyond a total).

Dimensions identified: dim_customer, dim_plan (Type 2 SCD — plan pricing/tier changes need history), dim_period (billing period, similar to dim_date at a coarser grain), plus a **factless fact table** for subscription lifecycle events (fact_subscription_event: signup, upgrade, downgrade, cancel — one row per event, no measures needed beyond the occurrence).

Table structures:

```
CREATE TABLE dim_plan (
    plan_sk          BIGINT PRIMARY KEY,
    plan_id          VARCHAR(50),
    plan_name        VARCHAR(100),
    plan_tier        VARCHAR(30),
    monthly_price    DECIMAL(10,2),
    is_current       BOOLEAN,
    effective_start  DATE,
    effective_end    DATE
);

CREATE TABLE fact_subscription_billing (
    billing_sk       BIGINT PRIMARY KEY,
    subscription_id  VARCHAR(50),      -- degenerate dimension
    customer_sk      BIGINT REFERENCES dim_customer(customer_sk),
    plan_sk          BIGINT REFERENCES dim_plan(plan_sk),
    period_sk        INT REFERENCES dim_period(period_sk),
    mrr_amount       DECIMAL(10,2),
    add_on_amount    DECIMAL(10,2),
    discount_amount  DECIMAL(10,2),
    days_active_in_period INT
);

-- Factless fact for lifecycle events
CREATE TABLE fact_subscription_event (
    event_sk         BIGINT PRIMARY KEY,
    subscription_id  VARCHAR(50),
    customer_sk      BIGINT,
    plan_sk          BIGINT,
    event_date_sk    INT,
    event_type       VARCHAR(30)      -- SIGNUP, UPGRADE, DOWNGRADE, CANCEL, REACTIVATE
);
```

Design decisions explained: Splitting the periodic **billing snapshot** (money) from the **event stream** (lifecycle) mirrors the periodic-snapshot vs. transaction-fact distinction from Book 1 — MRR is best measured as a regular per-period snapshot, while churn/upgrade analysis is best measured as discrete events. Trying to derive churn purely from gaps in the billing fact is fragile; an explicit CANCEL event row is unambiguous.

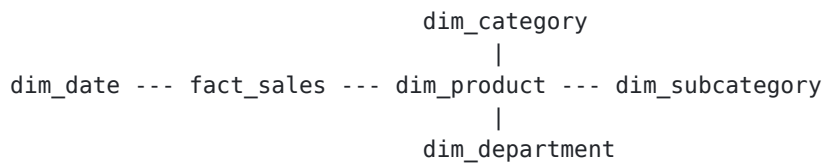
Practical scenario: *“Why model subscription lifecycle events (signup, upgrade, cancel) as a separate factless fact table instead of just inferring them from gaps in the billing fact table?”* **Approach:** Inferring events from billing gaps is fragile and ambiguous — a gap could mean cancellation, a billing failure, a data pipeline delay, or a plan change that temporarily changes the billing cycle. An explicit event fact table records the business event directly and unambiguously, at the moment it happens, and supports time-to-event and funnel analysis (e.g., time from signup to first upgrade) that a snapshot-only model can’t easily answer.

PART 6 — SNOWFLAKE SCHEMA

6.1 What is a Snowflake Schema?

What is it? A dimensional model where dimension tables are further **normalized** into multiple related tables — e.g., instead of one flat `dim_product`, you split it into `dim_product` -> `dim_subcategory` -> `dim_category` -> `dim_department`. Visually, the extra layers of joins radiating out from the fact table resemble a snowflake rather than a clean star.

Diagram:



(Compare to the flat Star Schema version in Part 5, where category, subcategory, and department are just columns directly on `dim_product`.)

6.2 How It Differs From Star Schema

	Star Schema	Snowflake Schema
Dimension structure	Flat, denormalized	Normalized into multiple related tables
Joins per query	Fewer	More (extra hops through normalized hierarchy)
Redundancy	Higher	Lower
Query simplicity	Higher	Lower (more joins to understand/write)
Storage	Slightly more (repeated attribute values)	Slightly less

6.3 Normalized Dimensions

Example — snowflaking the product hierarchy:

```

CREATE TABLE dim_department (
    department_sk BIGINT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE dim_category (
    category_sk BIGINT PRIMARY KEY,
    category_name VARCHAR(100),
    department_sk BIGINT REFERENCES dim_department(department_sk)
);

CREATE TABLE dim_product_snowflaked (
    product_sk BIGINT PRIMARY KEY,
  
```

```

    product_id    VARCHAR(50),
    product_name  VARCHAR(200),
    brand         VARCHAR(100),
    category_sk   BIGINT REFERENCES dim_category(category_sk)
);

```

A query needing the department name now requires two additional joins compared to the flat star version:

```

SELECT dep.department_name, SUM(f.line_amount) AS revenue
FROM fact_order_item f
JOIN dim_product_snowflaked p ON f.product_sk = p.product_sk
JOIN dim_category c ON p.category_sk = c.category_sk
JOIN dim_department dep ON c.department_sk = dep.department_sk
GROUP BY dep.department_name;

```

6.4 Advantages

- Less storage redundancy for large, deeply hierarchical dimensions.
- A hierarchy change (e.g., renaming a department, re-parenting a category) only touches one small table instead of rewriting every row of a large flat dimension.
- Can better match a genuinely normalized, centrally-governed master data hierarchy (e.g., an official chart of accounts, or a corporate product taxonomy managed by a separate team).

6.5 Disadvantages

- More joins per query — more complexity for business users and BI tools, and typically slightly worse query performance versus a flat star (though modern columnar engines like Snowflake handle this reasonably well).
- Harder to reason about and to document/govern than a flat star.

6.6 Performance Considerations

In Snowflake specifically, extra joins across normalized dimension tables are usually **not** the dominant cost for typical BI queries, since dimension tables are small relative to fact tables and Snowflake’s query optimizer handles multi-way joins on small tables efficiently. The bigger performance factor is almost always how effectively the **fact table** is pruned (via clustering/partition pruning), not the shape of the dimension side. That said, unnecessary snowflaking still adds real *query authoring and maintenance* complexity even if raw performance impact is modest.

6.7 Storage Considerations

Storage savings from snowflaking a dimension are usually small in absolute terms for cloud columnar warehouses, since columnar compression already handles repeated values (like a repeated category_name string) very efficiently. This means the classic “normalize to save disk space” argument is much weaker in Snowflake than it was in older row-oriented on-premise databases — governance and update-consistency arguments are usually the stronger justification for snowflaking today.

6.8 When to Use Star Schema

Default choice. Use a flat Star Schema when: - The dimension is small-to-medium and doesn’t change hierarchy structure often. - Business users and BI tools need to query it directly with minimal joins. - Storage savings from normalizing wouldn’t be meaningful (true for most cloud columnar warehouses).

6.9 When to Use Snowflake Schema

Use a Snowflake Schema (or snowflake *just one* hierarchy within an otherwise-flat star) when: - A hierarchy is centrally governed by a separate master-data domain and changes independently of the rest of the dimension (e.g., an official chart of accounts or corporate org hierarchy). - The hierarchy changes frequently enough that rewriting a large flat dimension on every change is a real operational cost. - Multiple fact tables/marts need to share the *exact same* governed hierarchy and a single small lookup table is the cleanest way to guarantee that consistency.

Common mistakes: snowflaking dimensions by default “because it’s more normalized/correct” — in a warehouse this usually just adds unnecessary query complexity for little real benefit given how cheap columnar storage handles denormalized redundancy; conversely, refusing to snowflake even a large, centrally-governed hierarchy (like finance’s chart of accounts) and ending up with inconsistent, hard-to-maintain flattened copies across multiple marts.

Practical scenario: *“Your BI team complains that a `dim_product` snowflaked into four tables is slowing down their dashboards and confusing their SQL. What would you recommend?”* **Approach:** Unless there’s a strong governance or update-frequency reason to keep the hierarchy normalized, flatten it back into a single denormalized `dim_product` (a “star” version) for the BI-facing semantic layer — this can even be done as a view or a downstream dbt model built on top of the normalized master tables, giving you the governance benefits of the normalized source while still presenting a simple flat dimension to consumers.

Practical scenario: *“Is storage savings still a good argument for choosing Snowflake Schema over Star Schema in a modern cloud warehouse like Snowflake?”* **Approach:** Generally weak — columnar compression already handles repeated denormalized values efficiently, so the storage savings from normalizing are usually marginal. The stronger, more current arguments for snowflaking are about governance (a hierarchy managed by a separate, authoritative source) and update consistency (avoiding having to rewrite a huge flat dimension on every hierarchy change), not raw disk space.

PART 7 — SLOWLY CHANGING DIMENSIONS (DEEP DIVE)

7.1 What is SCD and Why Historical Data Matters

What is it? Slowly Changing Dimensions are the standard techniques for handling dimension attributes that change over time — a customer moves cities, an employee changes department, a product gets recategorized — and deciding *how much history to preserve* when they do.

Why historical data matters: A data warehouse’s entire value proposition includes answering “what did things look like *then*,” not just “what do they look like now.” If a customer’s region is silently overwritten every time they move, a report of “last year’s sales by region,” re-run today, will incorrectly attribute all their historical purchases to their *current* region — the report would give a different answer each time it’s re-run, even though nothing about last year’s sales actually changed. That’s the core problem SCD techniques solve.

Real-world example: A telecom customer moves from the “Northeast” region to the “Southwest” region. Depending on the SCD type chosen, last year’s revenue report by region either stays historically accurate (Type 2) or silently changes (Type 1) the next time it’s run.

7.2 SCD Type 0 — Retain Original

Definition: The attribute is **never updated** once loaded — the original value is preserved forever, even if the source system’s value changes.

Use case: Attributes that are fundamentally immutable by business rule, or where the “original” value is what matters for the business (e.g., `original_signup_channel`, `date_of_birth` if the business treats it as fixed once recorded, `original_credit_score_at_application` for a loan).

Example — before/after:

Source system updates `date_of_birth` (data entry correction):

Before: `customer_id=101, date_of_birth=1990-05-14`

After: `customer_id=101, date_of_birth=1990-05-04` (corrected typo)

Type 0 dimension: unaffected — keeps 1990-05-14 forever

(unless a deliberate one-time data-quality correction is made outside normal SCD processing)

SQL implementation: Simplest of all types — load once, then explicitly **exclude** the attribute from any future update logic:

```
-- Initial load only; subsequent loads skip this column entirely for existing rows
INSERT INTO dim_customer (customer_sk, customer_id, original_signup_channel)
SELECT seq_customer_sk.NEXTVAL, customer_id, signup_channel
FROM stg_customer
WHERE customer_id NOT IN (SELECT customer_id FROM dim_customer);
-- No UPDATE statement ever touches original_signup_channel
```

Advantages: simplest possible implementation; guarantees a stable, auditable “as originally recorded” value. **Disadvantages:** by design, doesn’t reflect corrections — a genuine data-entry error will persist unless handled by a separate, deliberate correction process. **When to use:** attributes where the *original* value has specific business meaning, or truly immutable facts. **When NOT to use:** any attribute where the business actually wants to see updates/corrections

reflected.

Practical scenario: “When would you deliberately choose SCD Type 0, given it means never updating the value?” **Approach:** When the original value itself is the business-relevant fact — e.g., “which marketing channel originally acquired this customer” is meaningful regardless of any later re-classification, or a loan’s “credit score at time of application” needs to stay fixed for regulatory/audit reasons even if the customer’s credit score changes later.

7.3 SCD Type 1 — Overwrite

Definition: When the attribute changes, the existing dimension row is simply **updated in place** — no history is kept.

Use case: Correcting data entry errors, or attributes where only the current value matters and historical accuracy of past reports referencing the old value is not required (e.g., correcting a misspelled name).

Example — before/after:

Before: customer_sk=55, customer_id=C-101, email='old@example.com'

After: customer_sk=55, customer_id=C-101, email='new@example.com' (same row, overwritten)

SQL implementation:

```
UPDATE dim_customer d
SET email = s.email
FROM stg_customer s
WHERE d.customer_id = s.customer_id
AND d.email <> s.email;
```

Advantages: simple, no extra storage for historical rows, appropriate for genuine corrections. **Disadvantages:** historical fact rows joined to this dimension now reflect the *current* attribute value for *all* history — as shown in the telecom region example above, this can silently rewrite historical reports. **When to use:** correcting errors, or attributes genuinely not needed for historical trend analysis (e.g., a customer’s phone number, for most reporting purposes). **When NOT to use:** any attribute used to segment/group historical reporting where the business cares about “how things were classified at the time” (region, price tier, customer segment).

Practical scenario: “What’s the practical risk of using SCD Type 1 for a customer’s ‘sales region’ attribute?” **Approach:** Since Type 1 overwrites in place, all historical fact rows joined to that customer will reflect their *current* region after any update — meaning a “sales by region, trended over the last 2 years” report will silently reclassify all of that customer’s historical sales into their new region every time the report is re-run, which usually isn’t what stakeholders expect or want from historical trend analysis.

7.4 SCD Type 3 — Add a Column for Previous Value

Definition: Adds one (or a small fixed number of) extra column(s) to store the **immediately prior** value alongside the current value — limited history, but simple and queryable without extra joins.

Use case: When the business specifically needs to compare “before vs. after” a single, infrequent change (e.g., “current region” vs. “previous region” for a one-time territory realignment), but doesn’t need full unlimited history.

Example — before/after:

Before: customer_sk=55, current_region='Northeast', previous_region=NULL

After: customer_sk=55, current_region='Southwest', previous_region='Northeast'
-- A SECOND change would overwrite previous_region again, losing 'Northeast'

SQL implementation:

```
UPDATE dim_customer d
SET previous_region = d.current_region,
    current_region = s.region,
    region_change_date = CURRENT_DATE()
FROM stg_customer s
WHERE d.customer_id = s.customer_id
AND d.current_region <> s.region;
```

Advantages: simple before/after comparisons without joining to a history table; no row explosion. **Disadvantages:** only retains one (or a fixed number of) prior value(s) — a second change loses the first change’s history; doesn’t scale to attributes that change often or need unlimited history. **When to use:** rare, well-understood, largely one-time reclassifications (a sales territory realignment, a one-time product recategorization) where “before vs. after this specific event” is the actual business question. **When NOT to use:** attributes that change repeatedly over time and need full historical trend analysis — use Type 2 instead.

Practical scenario: *“Why is SCD Type 3 a poor fit for an attribute that changes frequently, like a stock’s price tier that gets reclassified every quarter?”* **Approach:** Type 3 only stores a single “previous value” column, so each new change overwrites the prior “previous” value — after several changes, all history except the most recent transition is lost. An attribute that changes repeatedly needs unlimited history, which is exactly what Type 2 (new row per change) provides.

7.5 SCD Type 2 — Add a New Row (Full History) — VERY DEEP DIVE

Definition: When a tracked attribute changes, the **existing dimension row is “expired”** (its end date is set, current flag turned off) and a **new row is inserted** with a new surrogate key, the new attribute value, a new effective start date, and marked current. The business/natural key stays the same across all versions; only the surrogate key and effective-date/current-flag columns differ.

Why it’s the default choice for most warehouse dimensions: It’s the only one of the six types that gives **complete, queryable history** while still letting fact tables join to “the dimension row that was true at the time of the fact” — which is exactly what most historically-accurate BI reporting needs.

Core columns required

Column	Purpose
<entity>_sk (surrogate key)	Unique per <i>version</i> of the entity — this is what fact tables reference.
<entity>_id (business/natural key)	Identifies the same real-world entity across all its versions.
effective_start_date	When this version became true.
effective_end_date	When this version stopped being true (or a far-future sentinel like 9999-12-31 if still current).
is_current	Boolean flag for quickly filtering “current” rows without a date comparison.

Example — before/after (customer moves region)

BEFORE the change:

customer_sk	customer_id	region	eff_start	eff_end	is_current
100	C-101	Northeast	2024-01-01	9999-12-31	TRUE

AFTER the change (customer moves 2026-03-15):

customer_sk	customer_id	region	eff_start	eff_end	is_current	
100	C-101	Northeast	2024-01-01	2026-03-14	FALSE	<- expired
101	C-101	Southwest	2026-03-15	9999-12-31	TRUE	<- new current row

Note: customer_id (business key) is the same on both rows; customer_sk (surrogate key) differs — this is precisely what lets a fact row created in 2025 correctly join to customer_sk=100 (Northeast) forever, while a fact row created after 2026-03-15 joins to customer_sk=101 (Southwest).

Full SQL implementation — MERGE-based Type 2 load

This is the standard production pattern, run incrementally (e.g., nightly) against a staging table of the latest source snapshot:

```
-- Step 1: Identify changed rows by comparing staging to the CURRENT dimension row
CREATE OR REPLACE TEMPORARY TABLE stg_customer_changes AS
SELECT s.customer_id, s.region, s.customer_segment, s.full_name
FROM stg_customer s
JOIN dim_customer d
  ON s.customer_id = d.customer_id
 AND d.is_current = TRUE
WHERE s.region <> d.region           -- tracked attribute changed
```

```

OR s.customer_segment <> d.customer_segment;

-- Step 2: Expire the current row for changed customers
UPDATE dim_customer d
SET effective_end_date = CURRENT_DATE() - 1,
    is_current = FALSE
FROM stg_customer_changes c
WHERE d.customer_id = c.customer_id
    AND d.is_current = TRUE;

-- Step 3: Insert new current rows for changed customers
INSERT INTO dim_customer (
    customer_sk, customer_id, full_name, region, customer_segment,
    effective_start_date, effective_end_date, is_current
)
SELECT
    dim_customer_sk_seq.NEXTVAL,
    c.customer_id, c.full_name, c.region, c.customer_segment,
    CURRENT_DATE(), '9999-12-31', TRUE
FROM stg_customer_changes c;

-- Step 4: Insert brand-new customers (never seen before)
INSERT INTO dim_customer (
    customer_sk, customer_id, full_name, region, customer_segment,
    effective_start_date, effective_end_date, is_current
)
SELECT
    dim_customer_sk_seq.NEXTVAL,
    s.customer_id, s.full_name, s.region, s.customer_segment,
    CURRENT_DATE(), '9999-12-31', TRUE
FROM stg_customer s
LEFT JOIN dim_customer d ON s.customer_id = d.customer_id
WHERE d.customer_id IS NULL;

```

Equivalent single-statement pattern using Snowflake's MERGE (common in dbt snapshot-style implementations — full dbt snapshot example in Book 3):

```

MERGE INTO dim_customer AS target
USING (
    SELECT s.*, d.customer_sk AS existing_sk, d.is_current AS existing_is_current
    FROM stg_customer s
    LEFT JOIN dim_customer d
        ON s.customer_id = d.customer_id AND d.is_current = TRUE
) AS source
ON target.customer_sk = source.existing_sk AND target.is_current = TRUE
WHEN MATCHED AND (source.region <> target.region
    OR source.customer_segment <> target.customer_segment) THEN
    UPDATE SET effective_end_date = CURRENT_DATE() - 1, is_current = FALSE
WHEN NOT MATCHED THEN
    INSERT (customer_sk, customer_id, full_name, region, customer_segment,
        effective_start_date, effective_end_date, is_current)
    VALUES (dim_customer_sk_seq.NEXTVAL, source.customer_id, source.full_name,
        source.region, source.customer_segment, CURRENT_DATE(), '9999-12-31', TRUE);

-- Note: MERGE alone can expire OR insert but the "expire-then-insert-new-version"
-- two-row outcome per change still typically needs the row insertion as a
-- separate follow-up INSERT (as in the multi-step pattern above), since a single
-- MERGE statement can't both update an existing row AND insert a second new row
-- for the same input row in one pass.

```

Incremental processing considerations

- **Change detection** should compare only the attributes you actually want tracked as Type 2 (region, segment) — not every column — otherwise trivial, unimportant changes (a reformatted phone number) trigger unnecessary new dimension rows. This is why real-world Type 2 pipelines maintain an explicit list of “Type 2 tracked columns” vs. “Type 1 tracked columns” per dimension (a **hybrid** approach — see Type 6, section 7.7).
- **Hashing for change detection** is common at scale: compute a hash of the tracked attribute columns and compare hashes instead of column-by-column, which is both faster and less error-prone as the column list grows:

```
SELECT customer_id, MD5(CONCAT(region, '|', customer_segment)) AS row_hash
FROM stg_customer;
```

Late-arriving changes

Problem: A change event arrives out of order — e.g., you learn today that a customer’s region actually changed *two weeks ago*, not today. Naively expiring “as of today” would misattribute two weeks of fact rows to the wrong dimension version.

Handling it correctly: 1. Determine the *true* effective date of the change (from the source event, not the load date). 2. Find the dimension row that was current *at that true effective date* (not necessarily the row currently flagged *is_current*). 3. Split that row’s date range at the true effective date: shorten the old row’s *effective_end_date*, insert the new row with *effective_start_date* = the true change date, and — if the changed row was not the most recent — also adjust the *next* row’s start date so ranges stay contiguous with no gaps or overlaps.

```
-- Simplified late-arriving Type 2 correction: insert a new version between two
-- existing versions, splitting the affected date range
UPDATE dim_customer
SET effective_end_date = '2026-03-01' -- the true change date - 1
WHERE customer_id = 'C-101'
  AND effective_start_date <= '2026-03-01'
  AND effective_end_date >= '2026-03-01';

INSERT INTO dim_customer (customer_sk, customer_id, region, effective_start_date, effective_end_date, is_current)
VALUES (dim_customer_sk_seq.NEXTVAL, 'C-101', 'Southwest', '2026-03-02', '9999-12-31', TRUE);
-- (with the previously-current row's end date also corrected if it's no longer the last segment)
```

This is genuinely one of the trickiest real-world SCD scenarios — it requires re-processing history rather than just appending, and any fact table joins done by *date range lookup* (rather than a static, cached surrogate key stored at fact-load time) will need to be re-evaluated for the affected period. Many pipelines simplify this by only supporting late-arriving corrections through a controlled backfill/reprocessing job rather than routine incremental logic (see Part 9, section 9.10).

Advantages of Type 2 overall: true, queryable, unlimited historical accuracy; the gold standard for dimensions used in trend/historical analysis. **Disadvantages:** more complex ETL/ELT logic; dimension tables grow larger (multiple rows per real-world entity); fact-to-dimension joins by date range (rather than a stored surrogate key) can be slower and more error-prone if not carefully implemented.

Common mistakes: tracking every single column as Type 2 (causing dimension row explosion for even trivial changes); forgetting the sentinel end date / current flag and instead relying only on *MAX(effective_start_date)* (fragile under late-arriving data); not handling late-arriving changes at all and simply always inserting “as of today,” which silently corrupts historical accuracy for the exact scenario Type 2 exists to solve.

Practical scenario: *“Walk me through exactly what happens in the dimension table when a tracked attribute changes under SCD Type 2.”* **Approach:** The current row for that business key is “expired” — its `effective_end_date` is set to the day before the change and `is_current` flips to `FALSE` — and a brand-new row is inserted with a new surrogate key, the updated attribute value, `effective_start_date` set to the change date, `effective_end_date` set to a far-future sentinel, and `is_current` = `TRUE`. The business key is identical on both rows; only the surrogate key, attribute value, and date range differ.

Practical scenario: *“How do late-arriving dimension changes complicate a standard SCD Type 2 pipeline?”* **Approach:** A late-arriving change means the “as of today” assumption incremental Type 2 logic normally relies on is wrong — the true effective date is in the past, potentially in the middle of an already-existing dimension row’s date range. Handling it correctly requires splitting that existing row’s range at the true change date rather than simply appending a new “current” row, and may require re-evaluating any fact rows in the affected period that were joined by date-range lookup rather than a stored surrogate key.

7.6 SCD Type 4 — History Table

Definition: Split the dimension into two physical tables: a **current** table (always reflects the latest state, Type 1 style, small and fast) and a separate **history** table (append-only log of every past version, similar in spirit to Type 2 but kept out of the main dimension).

Example structure:

```
CREATE TABLE dim_customer_current (
  customer_sk    BIGINT PRIMARY KEY,
  customer_id    VARCHAR(50),
  region         VARCHAR(50),
  customer_segment VARCHAR(50),
  last_updated   TIMESTAMP
);

CREATE TABLE dim_customer_history (
  history_sk     BIGINT PRIMARY KEY,
  customer_id    VARCHAR(50),
  region         VARCHAR(50),
  customer_segment VARCHAR(50),
  valid_from     TIMESTAMP,
  valid_to       TIMESTAMP
);
```

Advantages: the “current” table stays small and fast for the vast majority of queries (which only need current attributes), while full history is still available in a separate table for the minority of queries that need it; avoids bloating the primary dimension with many historical rows. **Disadvantages:** two tables to maintain; queries needing “as of a past date” context must explicitly join to the history table instead of the primary dimension, adding a bit of query complexity for historical analysis. **When to use:** very large dimensions where most queries only need current state and history is needed only occasionally (e.g., audit/compliance queries) — keeping history out of the hot path improves everyday query performance.

Practical scenario: “How is SCD Type 4 different from Type 2, and why might you choose it for a very large, frequently-changing dimension?” **Approach:** Type 2 keeps all historical versions in the *same* dimension table, which grows large and requires an `is_current` filter on every query that only cares about current state. Type 4 physically separates “current” (small, fast, Type-1-like) from “history” (a separate append-only log), so the common case — querying current attributes — stays fast, and the less common case — full historical lookups — explicitly opts into the larger history table instead of paying that cost by default.

7.7 SCD Type 6 — Hybrid (1 + 2 + 3)

Definition: Combines all three earlier techniques in one dimension row: keep full historical rows like Type 2 (new row per change, effective dates), **and** add a “current value” column (updated on every historical row, Type 1 style) so historical rows can also show what the attribute’s value eventually became, **and** optionally a “previous value” column (Type 3 style) for quick before/after comparison.

Example structure:

```
CREATE TABLE dim_customer_type6 (
  customer_sk    BIGINT PRIMARY KEY,
  customer_id    VARCHAR(50),
  region_historical VARCHAR(50), -- Type 2: value as of this row's period
  region_current  VARCHAR(50), -- Type 1: always the latest value, updated on ALL rows for this customer_id
  region_previous VARCHAR(50), -- Type 3: value immediately prior to region_historical
```

```

    effective_start_date DATE,
    effective_end_date   DATE,
    is_current           BOOLEAN
);

```

Why this is powerful: A single row lets you answer *both* “what was true at the time of this historical fact” (region_historical) *and* “what is this customer’s region today, even when looking at an old fact row” (region_current) — without needing a second join to the current dimension row. This is genuinely useful for reports like “show historical orders, but group by the customer’s *current* segment” (a common real request — e.g., “how much revenue have our *currently* VIP customers generated historically, even from before they became VIP”).

SQL implementation — maintaining region_current across all historical rows on every change:

```

-- Step 1 & 2: same as standard Type 2 – expire current row, insert new version
-- (omitted here, identical to section 7.5)

-- Step 3: propagate the new current value back onto ALL historical rows
-- for this customer_id (this is the Type 6-specific extra step)
UPDATE dim_customer_type6
SET region_current = (
    SELECT region_historical FROM dim_customer_type6 t2
    WHERE t2.customer_id = dim_customer_type6.customer_id AND t2.is_current = TRUE
)
WHERE customer_id = 'C-101';

```

Advantages: most flexible single-table option — supports “as-of” historical analysis and “current-context” historical analysis simultaneously without extra joins. **Disadvantages:** most complex to implement and maintain of all SCD types; the extra UPDATE touching all historical rows for a changed entity adds ETL cost, and the design is easy to get subtly wrong (e.g., forgetting to propagate region_current correctly). **When to use:** when business reporting genuinely needs both “as it was then” and “as it is now” perspectives on the same dimension, e.g., cohort analysis segmented by current status. When only one perspective is ever needed, plain Type 1 or Type 2 is simpler and sufficient.

Practical scenario: “A business stakeholder wants to see all-time historical revenue for customers who are currently in the VIP segment — even revenue from before they became VIP. What SCD design supports this cleanly?” **Approach:** SCD Type 6 — its region_current/segment_current-style column (kept in sync across all historical rows for that business key) lets a query filter by *current* segment while still joining to historical fact rows correctly, without needing a separate join back to the latest dimension row. A pure Type 2 dimension would require an extra self-join to the current row to get this same answer.

Type	New Row on Change?	Keeps History?	Complexity	Typical Use
------	--------------------	----------------	------------	-------------

7.8 SCD Type Comparison Summary

Type	New Row on Change?	Keeps History?	Complexity	Typical Use
0	No	Original value only	Very low	Immutable/original-value attributes
1	No (overwrite)	No	Low	Corrections, non-analytical attributes
2	Yes	Full, unlimited	Medium-High	Default for analytically important attributes
3	No (extra column)	One prior value	Low-Medium	Rare one-time before/after comparisons
4	Yes, in separate table	Full, in history table	Medium	Very large dims, current-state-heavy workloads
6	Yes + propagated current	Full + always-current view	High	Need both “as-of” and “current-context” views

Practical scenario: “How would you decide which SCD type to apply to a given dimension attribute in practice?” **Approach:** Ask whether historical accuracy actually matters for that specific attribute — if not (e.g., fixing a typo), Type 1. If yes, and the business only ever needs “as it was at the time,” Type 2. If both “as it was” and “as it is now” views are needed simultaneously, Type 6. If the dimension is enormous and most queries only need current state, split current vs. history physically with Type 4. In practice, most production dimensions use a **mix** — some columns tracked as Type 1 (corrections, cosmetic fields), others as Type 2 (segment, region, tier) — rather than applying one SCD type uniformly to an entire table.

PART 8 — ADVANCED DATA MODELING CONCEPTS

8.1 Hierarchies, Parent-Child, and Recursive Relationships

What is it? A hierarchy is a series of one-to-many relationships forming levels (department -> category -> product). A **parent-child** or **recursive relationship** is when an entity relates to *itself* at a variable, often unknown-in-advance depth — the classic example is an employee/org chart, where an employee has a manager who is also an employee.

Table example (recursive):

```
CREATE TABLE dim_employee (
  employee_sk BIGINT PRIMARY KEY,
  employee_id VARCHAR(50),
  employee_name VARCHAR(200),
  manager_sk BIGINT REFERENCES dim_employee(employee_sk) -- self-reference
);
```

SQL example — recursive CTE to flatten an org hierarchy:

```
WITH RECURSIVE org_chart AS (
  SELECT employee_sk, employee_name, manager_sk, 1 AS depth
  FROM dim_employee
  WHERE manager_sk IS NULL -- top of the org

  UNION ALL

  SELECT e.employee_sk, e.employee_name, e.manager_sk, o.depth + 1
  FROM dim_employee e
  JOIN org_chart o ON e.manager_sk = o.employee_sk
)
SELECT * FROM org_chart ORDER BY depth;
```

Why it matters: Fixed-depth hierarchies (department -> category -> product) can be flattened into columns on a dimension (as in the Star Schema examples). **Variable-depth** hierarchies (an org chart with an unknown number of management levels) cannot — they require either a recursive/self-referencing model, or a pre-computed “bridge”/closure table (below) if the analytical engine doesn’t support recursive queries well or if performance of recursive queries is a concern at scale.

Common mistakes: trying to flatten a variable-depth hierarchy into fixed columns (level_1, level_2, level_3, ...) — this breaks the moment the hierarchy is deeper than anticipated, and is a direct parallel to the multi-valued-attribute mistake from Book 1.

Practical scenario: “How would you model an organizational hierarchy where the number of management levels varies by department?” **Approach:** A self-referencing (parent-child) table with a manager_sk foreign key pointing back to the same dimension, queried with a recursive CTE (or a precomputed closure/bridge table for performance at scale) — rather than flattening into a fixed number of level columns, which breaks whenever actual depth exceeds what was anticipated.

8.2 Many-to-Many Relationships & Bridge Tables

What is it? Covered at the OLTP level in Book 1 (junction tables); in a **dimensional** model, the equivalent construct is a **bridge table** connecting a fact (or a dimension) to a *group* of related dimension members, since a single foreign key can't represent "many."

Real-world example (banking — joint accounts): One account can have multiple customers (joint holders); one customer can have multiple accounts.

Table example:

```
CREATE TABLE bridge_account_customer (
  account_sk    BIGINT REFERENCES dim_account(account_sk),
  customer_sk   BIGINT REFERENCES dim_customer(customer_sk),
  ownership_pct DECIMAL(5,2),  -- optional weighting factor
  PRIMARY KEY (account_sk, customer_sk)
);
```

Why the weighting factor matters: Without ownership_pct (or similar), summing a fact (e.g., account balance) across the bridge for "total balance by customer" would **double-count** joint-account balances once per joint holder. A weighting factor lets you allocate the fact proportionally instead of the naive (and wrong) full-double-count.

```
-- Weighted allocation avoids double-counting joint account balances per customer
SELECT c.customer_sk, SUM(f.balance * b.ownership_pct) AS allocated_balance
FROM fact_daily_account_balance f
JOIN bridge_account_customer b ON f.account_sk = b.account_sk
JOIN dim_customer c ON b.customer_sk = c.customer_sk
GROUP BY c.customer_sk;
```

Common mistakes: joining a fact table straight through a many-to-many bridge without a weighting factor and without acknowledging the fan-out risk — this silently inflates any additive measure.

Practical scenario: "A joint bank account has two customers. If you join your fact table to a bridge table connecting accounts to customers, what happens to the account's balance in a naive SUM() by customer?" **Approach:** It gets double-counted — the balance fact row joins to both customers via the bridge, so summing balance grouped by customer would count that account's full balance once for *each* joint holder, overstating totals if you then sum across customers. The fix is either an explicit weighting/allocation factor in the bridge table, or being deliberate that the query answers "which customers have exposure to this balance" rather than "what is the true total balance across distinct customers."

8.3 Late-Arriving Dimensions and Facts

Late-arriving dimension: A fact arrives referencing a dimension member (e.g., a customer_id) that doesn't exist yet in the dimension table — common when source systems load asynchronously or a customer record is created slightly after their first transaction is recorded.

Late-arriving fact: A fact event is received well after the period it actually occurred in (e.g., a delayed batch feed delivers last month's transactions today) — this affects which dimension row (which SCD Type 2 version) it should correctly join to.

Handling late-arriving dimensions — the "inferred member" pattern:

```
-- When a fact references a customer_id not yet in dim_customer,
-- insert a minimal placeholder row rather than dropping the fact
INSERT INTO dim_customer (customer_sk, customer_id, full_name, region, is_current, is_inferred_member)
SELECT dim_customer_sk_seq.NEXTVAL, f.customer_id, 'Unknown (Inferred)', 'Unknown', TRUE, TRUE
```

```

FROM stg_fact f
LEFT JOIN dim_customer d ON f.customer_id = d.customer_id
WHERE d.customer_id IS NULL;
-- Later, when the real customer record arrives, this placeholder row is
-- updated in place (Type 1 style) with the real attributes, and
-- is_inferred_member is flipped to FALSE.

```

Why it matters: Dropping facts that reference not-yet-known dimension members loses real business events (lost revenue in a report); the inferred-member pattern preserves the fact and its measures immediately, backfilling descriptive context once it becomes available, without ever needing to reprocess the fact table itself.

Practical scenario: “An order fact arrives referencing a customer_id your dim_customer table doesn’t have yet. What do you do?” **Approach:** Insert a minimal placeholder (“inferred member”) row into dim_customer with the known business key and placeholder/unknown descriptive attributes, so the fact table isn’t blocked or dropped, then update that placeholder row in place once the real customer dimension data arrives — this way the fact’s measures are never lost and the join is corrected without touching the fact table.

8.4 Unknown Members

What is it? A dedicated, reserved dimension row (often surrogate key -1 or 0) representing “no value” / “not applicable” / “not yet known” — used so fact rows with a missing or invalid dimension reference can still join successfully (avoiding accidental row loss from an INNER JOIN, or NULL-related aggregation surprises).

Example:

```

INSERT INTO dim_customer (customer_sk, customer_id, full_name, region, is_current)
VALUES (-1, 'UNKNOWN', 'Unknown Customer', 'Unknown', TRUE);

-- Fact load maps missing/invalid customer_id to the unknown member instead of NULL
INSERT INTO fact_order_item (customer_sk, ...)
SELECT COALESCE(d.customer_sk, -1), ...
FROM stg_order_item s
LEFT JOIN dim_customer d ON s.customer_id = d.customer_id AND d.is_current = TRUE;

```

Why it matters: Without an unknown member, a NULL foreign key either breaks an INNER JOIN (silently dropping that fact row from reports) or requires every downstream query to remember to LEFT JOIN and handle NULLs specially. A dedicated unknown-member row makes “missing/invalid dimension reference” an explicit, queryable, first-class category instead of a silent gap.

Practical scenario: “Why use a dedicated ‘Unknown Member’ dimension row instead of just leaving the foreign key NULL?” **Approach:** A NULL foreign key breaks a standard INNER JOIN (silently excluding that fact row from any report using an inner join to the dimension) and forces every query author to remember to handle NULLs specially. A reserved “Unknown Member” row lets all fact rows join successfully and consistently under any join type, and makes “how much revenue has an unknown/invalid customer” an explicit, easily reportable category rather than a silent gap.

8.5 Multi-Valued Attributes

What is it? An attribute where a single dimension member can have more than one simultaneous value — e.g., a product that belongs to multiple categories, a customer with multiple interests/tags.

Why it's tricky: It doesn't fit a simple one-to-one dimension attribute; representing it directly would either force choosing just one value (losing information) or require a bridge table (as in 8.2), which introduces fan-out risk for any fact joined through it.

Example (product in multiple categories) — bridge table:

```
CREATE TABLE bridge_product_category (  
    product_sk BIGINT REFERENCES dim_product(product_sk),  
    category_sk BIGINT REFERENCES dim_category(category_sk),  
    PRIMARY KEY (product_sk, category_sk)  
);
```

Practical scenario: “A product can belong to multiple categories in your catalog. How does this affect a simple `dim_product.category` column, and how would you handle it?”

Approach: A single flat category column can't represent multiple simultaneous categories without picking just one (losing information) or concatenating values (breaking easy filtering/grouping). The standard fix is a bridge table between `dim_product` and `dim_category`, being mindful that joining fact tables through it can fan out a product's sales across each of its categories — appropriate for “sales by category” (where a product's revenue legitimately shows up under each category it belongs to) but requiring care for “total sales” style queries to avoid double-counting.

8.6 Hierarchical Dimensions (Summary)

What is it? A dimension whose attributes naturally form levels of increasing granularity (department > category > subcategory > product) — the recurring theme across sections 6 (Snowflake Schema), 8.1 (recursive/variable-depth), and the running examples in Part 5.

Two implementation choices: 1. **Fixed-depth hierarchy:** flatten into columns on a single denormalized dimension (Star Schema style) — simplest, works when depth is known and stable. 2. **Variable-depth hierarchy:** self-referencing parent-child table with recursive queries or a precomputed closure/bridge table — required when depth varies (org charts, arbitrary category trees).

Practical scenario: “How do you decide whether to flatten a hierarchy into dimension columns versus modeling it as a self-referencing parent-child table?” **Approach:**

If the number of levels is fixed and well-known in advance (e.g., a retail category structure is always exactly department > category > subcategory), flattening into columns on a denormalized dimension is simpler and faster to query. If the depth genuinely varies per branch (an org chart, a bill-of-materials, a folder structure), a self-referencing parent-child model with recursive queries (or a precomputed closure table for performance) is required, since a fixed number of level columns can't represent arbitrary depth.

PART 9 — DATA MODELING FOR ETL/ELT

9.1 Source System Analysis & Source-to-Target Mapping

What is it? Before building any pipeline, understanding each source system’s own model (its keys, its update patterns, its quirks) and explicitly documenting how each source field maps to the target warehouse model.

Source-to-target mapping example (table):

Source Field	Source System	Transformation	Target Field	Target Table
cust_id	CRM (Salesforce)	Direct copy, cast to VARCHAR	customer_id	dim_customer
dob	CRM	Parse MM/DD/YYYY string to DATE	date_of_birth	dim_customer
region_cd	ERP (SAP)	Lookup via region_code_map	region	dim_customer
order_amt	E-commerce app DB	Divide by 100 (stored in cents)	line_amount	fact_order_item

Why it matters: This document becomes the single reference for what “the model” actually means in terms of the messy reality of source systems — and it’s what a data quality bug gets traced back against (“the mapping says X, the pipeline does Y — which is wrong?”).

Common mistakes: building the pipeline logic straight into code/SQL without documenting the mapping anywhere durable — this makes onboarding new engineers and auditing data lineage far harder than it needs to be.

9.2 Data Profiling, Identifying Business Keys, Identifying Grain

Data profiling: Before modeling, actually query the source data to understand it — null rates, distinct value counts, actual min/max ranges, whether a supposedly unique column really is unique.

```
-- Quick profiling pass on a new source table
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT customer_id) AS distinct_customers,
  COUNT(*) - COUNT(customer_id) AS null_customer_ids,
  MIN(signup_date) AS earliest_signup,
  MAX(signup_date) AS latest_signup
FROM stg_customer;
```

Identifying business keys: Don’t assume — verify. A column named id might not actually be unique per real-world entity (e.g., duplicated across a merge of two source systems).

```
-- Verify a candidate business key is actually unique
SELECT customer_id, COUNT(*)
FROM stg_customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Identifying grain: Apply the same discipline as Book 1, section 4.4 — but now grounded in what the *source data* actually supports. If the source system only provides daily aggregated exports (not individual transactions), your fact table’s grain is constrained to daily, not transactional, no matter what the “ideal” grain would be.

Practical scenario: “Why should you always profile a new source table before designing the target model, even if you already have a source-to-target mapping document from the business team?” **Approach:** Business documentation often describes the *intended* design, not the actual data as it exists — profiling catches reality gaps like unexpectedly high null rates, a supposedly-unique key that isn’t, or date ranges that don’t match expectations, all of which directly affect grain and key decisions in the target model. Designing purely from documentation risks building a model that breaks the moment real data is loaded.

9.3 Full Load vs Incremental Load

Full load: Re-extract and reload the entire source table every run. Simple, but doesn’t scale as source tables grow, and loses any change history not present in the current full snapshot.

Incremental load: Extract only new/changed rows since the last run (using a watermark, a modified_at column, or CDC), then merge/append into the target.

	Full Load	Incremental Load
Simplicity	High	Lower — needs change tracking
Scales to large sources	Poor	Good
Captures deletes automatically?	Yes (if comparing full snapshots)	Only if explicitly handled
Typical use	Small reference/lookup tables	Large, high-volume fact sources

Practical scenario: “When would full load actually still be the right choice over incremental load?” **Approach:** For small, low-volume reference/lookup tables (e.g., a country codes table, a small product catalog) where the full table is cheap to re-extract every run, full load avoids the added complexity of watermark/CDC tracking and automatically handles deletes (a row simply disappears from the next full snapshot), which incremental logic must otherwise handle explicitly.

9.4 CDC (Change Data Capture)

What is it? Capturing individual insert/update/delete events from a source system as they happen — typically by reading the database’s transaction/replication log — rather than periodically comparing full snapshots.

Diagram:

Source DB Transaction Log
|

```

CDC Connector (e.g., Debezium, Fivetran, Snowflake's native CDC via Streams on external tables)
|
v
Change Event Stream (INSERT/UPDATE/DELETE + before/after values + timestamp)
|
v
Applied incrementally to target (Bronze/Staging), then MERGE'd into the model

```

Why it matters: CDC captures **deletes** and **intermediate updates** that a simple “extract rows modified since last run” approach can miss (e.g., a row that was updated twice between runs — a snapshot-diff approach only sees the final state, CDC sees both changes).

Practical scenario: “Why is CDC generally preferred over a simple ‘extract rows where `modified_at > last_run_time`’ approach for high-value operational data?” **Approach:** A `modified_at`-based incremental extract only sees the *latest* state of each row since the last run — it misses intermediate changes (a value that changed twice) and typically can’t detect deletes at all (a deleted row simply vanishes with no `modified_at` to filter on). CDC captures every individual change event, including deletes, directly from the transaction log, giving a complete and accurate change history.

9.5 MERGE / UPSERT

What is it? A single SQL operation that inserts new rows and updates existing rows in one statement, based on a matching key — the standard mechanism for applying incremental/CDC changes into a target table.

```

MERGE INTO dim_product AS target
USING stg_product_changes AS source
ON target.product_id = source.product_id
WHEN MATCHED THEN
    UPDATE SET product_name = source.product_name,
               category = source.category,
               unit_cost = source.unit_cost
WHEN NOT MATCHED THEN
    INSERT (product_id, product_name, category, unit_cost)
    VALUES (source.product_id, source.product_name, source.category, source.unit_cost);

```

Practical scenario: “Why is MERGE preferred over separate `DELETE+INSERT` or `UPDATE+INSERT` statements for applying incremental changes?” **Approach:** MERGE performs the match-then-update-or-insert logic atomically in a single statement, avoiding race conditions and partial-failure states that separate statements could leave the table in, and is typically better optimized by the query engine (Snowflake included) than the equivalent multi-statement logic.

9.6 Watermarking

What is it? Tracking the highest value of some incrementing column (a timestamp, an auto-increment ID) processed so far, so the next incremental run knows exactly where to resume from.

```

-- Store the watermark after each successful run
CREATE TABLE etl_watermark (
    source_table VARCHAR(100) PRIMARY KEY,
    last_processed_value TIMESTAMP
);

```

```
-- Next run's extraction query uses the stored watermark
SELECT * FROM source_table
WHERE modified_at > (SELECT last_processed_value FROM etl_watermark WHERE source_table = 'orders');
```

Common mistakes: using a watermark column that isn't strictly increasing/reliable (e.g., a `modified_at` that can be backdated by manual data corrections), which can cause rows to be silently skipped; not handling clock skew between the source system and the extraction process, which can cause a narrow window of rows to be missed right at the watermark boundary.

Practical scenario: “What’s a common failure mode with timestamp-based watermarking, and how do you mitigate it?” **Approach:** Clock skew or slightly-delayed commits can mean a row with a `modified_at` at just before the watermark actually commits *after* the extraction query already ran, causing it to be silently skipped in that run and the next. A common mitigation is using a small overlap/safety buffer (re-extracting a short window before the watermark and de-duplicating) rather than an exact boundary cutoff.

9.7 Deduplication

What is it? Removing duplicate rows introduced by upstream retries, overlapping incremental extraction windows, or genuine source-system duplicates, typically keeping the latest/most authoritative version per business key.

```
-- Keep only the latest row per business key, using ROW_NUMBER
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY customer_id ORDER BY modified_at DESC
  ) AS rn
  FROM stg_customer
) WHERE rn = 1;
```

Practical scenario: “Your incremental extraction uses an overlapping safety window (from the watermarking mitigation above) to avoid missing rows. What extra step does this require downstream?” **Approach:** Explicit deduplication before merging into the target — since the overlap window will re-extract some rows already processed in the previous run, a `ROW_NUMBER()`-based (or similar) dedup step, keyed on the business key and ordered by the most recent/authoritative timestamp, must run before the `MERGE` to avoid inserting/updating with stale duplicate data.

9.8 Data Reconciliation

What is it? Periodically verifying that the warehouse’s data matches the source system’s — row counts, sums of key measures, or spot-check comparisons — to catch silent pipeline bugs (a filter that accidentally excludes valid rows, a join that fans out).

```
-- Simple reconciliation check: row counts and a key sum, source vs warehouse
SELECT 'source' AS system, COUNT(*) AS row_count, SUM(order_amount) AS total_amount
FROM source_system.orders
UNION ALL
SELECT 'warehouse', COUNT(*), SUM(line_amount)
FROM warehouse.fact_order_item;
```

Practical scenario: “How would you catch a silent fan-out bug (a join accidentally duplicating rows) before it reaches a business dashboard?” **Approach:** Automated reconciliation checks comparing aggregate totals (row counts, sums of key additive mea-

tures) between the source system and the corresponding warehouse fact table, run as part of the pipeline's CI/CD or scheduled as a data quality job — a fan-out bug typically shows up as an unexplained jump in row count or total relative to source, which a reconciliation check catches before it's visible in a dashboard.

9.9 Schema Evolution

What is it? Handling changes to the source system's schema over time (new columns added, columns renamed/removed, data type changes) without breaking the pipeline or silently losing data.

Common approaches: schema-on-read for raw/staging layers (accept whatever arrives, often via semi-structured VARIANT columns in Snowflake), explicit schema versioning/contracts for anything feeding the curated/mart layers, and automated alerts when an unexpected new column appears in a source extract.

Practical scenario: *"A source system adds a new column mid-quarter without notice. How should a well-designed pipeline handle this?"* **Approach:** Ideally, the raw/staging layer is schema-flexible enough (e.g., landing data as semi-structured JSON/VARIANT, or a staging table designed to tolerate additive changes) that it doesn't break outright, while an automated schema-drift detection step alerts the team to the new column so it can be deliberately incorporated into the curated model rather than silently ignored or causing a hard failure.

9.10 Late-Arriving Data and Backfills

Late-arriving data: Facts or dimension changes that arrive well after the period they logically belong to (a delayed batch feed, a corrected record). Covered in dimension-specific detail in Part 7 (SCD) and Part 8 (late-arriving facts/dimensions) — here, the pipeline-design angle: incremental pipelines need an explicit strategy for re-opening and reprocessing already-loaded periods, not just appending forward.

Backfill: A deliberate, often one-time reprocessing of historical data — e.g., after a bug fix, or when a new attribute needs to be populated retroactively across all history.

Common pattern — idempotent backfill via full period reprocessing:

```
-- Reprocess a specific historical period cleanly: delete then reinsert
-- (only safe if the fact table's grain and partition boundaries are well understood)
DELETE FROM fact_order_item WHERE date_sk BETWEEN 20260101 AND 20260131;

INSERT INTO fact_order_item
SELECT ... FROM stg_order_item_reprocessed
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31';
```

Practical scenario: *"What design choice makes backfills safe to re-run without creating duplicate or inconsistent data?"* **Approach:** Idempotency (see 9.11) — typically achieved by scoping the backfill to a well-defined partition/period and doing a full delete-then-insert (or MERGE) for that exact scope, so running the same backfill twice produces the same end state rather than accumulating duplicates.

9.11 Idempotency

What is it? The property that running the same pipeline/job multiple times with the same input produces the same result as running it once — no duplicate rows, no double-counted measures, regardless of retries or reruns.

Why it matters: Pipeline failures and retries are inevitable at scale; without idempotency, a retried job after a partial failure can silently duplicate data.

Common idempotent patterns: MERGE/UPSERT instead of blind INSERT; delete-then-insert scoped to an exact, well-defined partition/period before reloading; using a unique constraint (enforced via a test, since Snowflake doesn't enforce it at the DDL level) combined with MERGE to guarantee no duplicates even under retry.

Practical scenario: *“Why is idempotency considered a core design requirement for production ETL/ELT pipelines, not just a nice-to-have?”* **Approach:** Failures, timeouts, and retries are a normal part of running pipelines at scale (network blips, warehouse timeouts, orchestrator retries) — without idempotency, any retry after a partial failure risks silently duplicating rows or double-applying changes, corrupting downstream metrics in a way that's often hard to detect until a stakeholder notices inflated numbers. Idempotent design (MERGE-based upserts, scoped delete-then-insert) makes retries safe by default.

9.12 Designing a Model When Source Data is Messy

A practical checklist for messy sources: 1. **Profile first** (9.2) — never assume documented structure matches reality. 2. **Land raw, unmodified data in staging** — preserve the original as delivered, even if malformed, so nothing is lost if a downstream transformation turns out wrong. 3. **Define the grain explicitly**, even if the source data doesn't cleanly support it yet — document the gap and design staging transformations to close it (e.g., deduplicating to the intended grain). 4. **Build explicit, automated data quality tests** (not-null, uniqueness, referential integrity, accepted value ranges) rather than assuming the source is clean. 5. **Use the “unknown member” and “inferred member” patterns** (Part 8) rather than silently dropping rows with missing or invalid keys. 6. **Version/track schema changes** rather than letting silent drift break curated models. 7. **Reconcile against source regularly** (9.8) to catch drift or bugs early.

Practical scenario: *“You're handed a source feed with inconsistent date formats, occasional duplicate records, and a business key that's sometimes null. What's your overall approach to modeling from this?”* **Approach:** Land the data as-is in a staging layer first (nothing is lost, and it's available for debugging), then apply deliberate, tested transformations: standardize date parsing explicitly (handling multiple formats), deduplicate to the intended grain using a ROW_NUMBER() -style pattern keyed on the business key with a clear tie-breaking rule, and route rows with a null/invalid business key to an “unknown member” or a quarantine table rather than silently dropping them — all backed by automated data quality tests so any regression in source data quality is caught immediately rather than discovered downstream in a report.

Glossary — Book 2 Terms

Term	Definition
Backfill	A deliberate, often one-time reprocessing of historical data.
Bridge Table	A table connecting a fact or dimension to a group of related dimension members, resolving a many-to-many relationship.
CDC	Change Data Capture — capturing individual insert/update/delete events from a source system's transaction log.
Closure Table	A precomputed table listing all ancestor-descendant pairs in a variable-depth hierarchy, for fast querying without recursion.
Data Reconciliation	Verifying warehouse data matches source system data via counts/sums/spot-checks.
Deduplication	Removing duplicate rows, typically keeping the latest/most authoritative version per business key.
Idempotency	The property that re-running a pipeline with the same input produces the same result as running it once.
Inferred Member	A placeholder dimension row created when a fact references a not-yet-known dimension member.
Late-Arriving Dimension	A fact referencing a dimension member that doesn't exist in the dimension table yet.
Late-Arriving Fact	A fact event received well after the period it actually occurred in.
MERGE / UPSERT	A single SQL operation that inserts new rows and updates existing rows based on a matching key.
Multi-Valued Attribute	An attribute where a single entity can have more than one simultaneous value.
Parent-Child / Recursive Relationship	A relationship where an entity relates to itself at variable depth (e.g., an org chart).
SCD Type 0	Never update — original value retained forever.
SCD Type 1	Overwrite the existing value in place; no history kept.
SCD Type 2	Add a new row (new surrogate key, new effective dates) on change; full history kept.
SCD Type 3	Add an extra column for the previous value; limited (usually one-level) history.
SCD Type 4	Split current state and history into two separate physical tables.

Term	Definition
SCD Type 6	Hybrid of Types 1, 2, and 3 in a single dimension design.
Schema Evolution	Handling source schema changes over time without breaking the pipeline.
Snowflake Schema	A dimensional model where dimension tables are further normalized into related sub-tables.
Unknown Member	A reserved dimension row (e.g., surrogate key -1) representing missing/not-applicable values.
Watermarking	Tracking the highest processed value of an incrementing column to resume incremental extraction.

Book 2 Practical Scenarios (15 Scenarios)

1. Why is a Star Schema generally preferred over a fully normalized model for BI dashboards? *Answer:* It minimizes joins per query (one hop from fact to any dimension attribute), which is both faster for the query engine and far easier for business users and BI tools to work with, at the cost of some redundancy that columnar warehouses handle efficiently anyway.

2. When would you choose to snowflake a dimension rather than keep it as a flat star? *Answer:* When a hierarchy is centrally governed by a separate master-data domain, changes frequently enough that rewriting a large flat dimension is a real operational cost, or must be shared consistently across multiple fact tables/marts — not for storage savings alone, since columnar compression already handles denormalized redundancy efficiently.

3. Explain SCD Type 2 in your own words, including what changes in the dimension table when an attribute changes. *Answer:* The current row for that business key is expired (end date set, `is_current` flipped to false), and a new row is inserted with a new surrogate key, the updated value, a new start date, and `is_current = true`. The business key stays the same across versions; only the surrogate key and date range/flag differ.

4. Why do fact tables joined to an SCD Type 2 dimension by surrogate key preserve historical accuracy, while SCD Type 1 doesn't? *Answer:* Type 2 creates a new surrogate key per version, so a fact row loaded before a change keeps referencing the old (correct-for-its-time) surrogate key/row. Type 1 has only one row (one surrogate key) per business key that's overwritten in place, so all historical fact rows referencing that key reflect the current value after any update.

5. What's the difference between SCD Type 4 and SCD Type 2? *Answer:* Type 2 keeps all versions in a single table (filtered by `is_current` for the latest state). Type 4 physically separates a small "current" table (fast, Type-1-like) from a separate append-only "history" table, so everyday queries needing only current state don't have to filter through historical rows.

6. Give a scenario where SCD Type 6 is genuinely necessary rather than just "nice to have." *Answer:* When business reporting needs both "as it was at the time" and "as it is now" perspectives simultaneously — e.g., all-time historical revenue attributed by a customer's *current* VIP segment, even revenue from before they became VIP — which Type 6's propagated "current value" column supports without an extra self-join.

7. Why is defining the grain considered foundational even when designing for a specific ETL/ELT pipeline (not just the model in isolation)? *Answer:* The grain constrains what the source data must support and how the incremental/CDC logic needs to be scoped — e.g., you can't build a per-transaction grain fact table if the source only ever exports daily aggregates, and mismatched grain assumptions between pipeline and model are a common source of double-counted or missing metrics.

8. What's the key difference between CDC and a simple 'modified_at > last_run' incremental extraction approach? *Answer:* CDC reads the source's transaction log and captures every individual insert/update/delete event, including deletes and intermediate updates. A `modified_at`-based approach only sees the latest state as of extraction time and typically can't detect deletes at all.

9. Why is idempotency considered a core requirement for production pipelines rather than a nice-to-have? *Answer:* Failures and retries are a normal part of operating pipelines at

scale; without idempotent design (MERGE-based upserts, scoped delete-then-insert), a retry after a partial failure can silently duplicate rows or double-apply changes, corrupting downstream metrics.

10. How would you handle a fact row that references a dimension member (say, a customer) that doesn't exist yet in the dimension table? *Answer:* Insert a minimal “inferred member” placeholder row into the dimension with the known business key and placeholder attributes, so the fact isn't dropped or blocked, then update that row in place once real dimension data arrives.

11. Why use a dedicated ‘Unknown Member’ row instead of a NULL foreign key in a fact table? *Answer:* A NULL foreign key breaks standard INNER JOINS (silently excluding that fact row from reports) and forces every downstream query to remember to handle NULLs. A reserved unknown-member row lets all facts join consistently and makes “missing/invalid reference” an explicit, reportable category.

12. What problem does a bridge table solve, and what's the risk of using one without a weighting factor? *Answer:* A bridge table resolves many-to-many relationships between a fact/dimension and a group of related members (e.g., joint account holders). Without a weighting/allocation factor, summing a fact through the bridge (e.g., account balance by customer) double-counts the shared amount once per related member.

13. How would you model a hierarchy with a variable, unknown-in-advance number of levels (e.g., an org chart)? *Answer:* A self-referencing parent-child table (e.g., `manager_sk` pointing back to the same dimension), queried with a recursive CTE, or a precomputed closure/bridge table for performance — not a fixed number of level columns, which breaks whenever actual depth exceeds what was anticipated.

14. Why should raw/staging data always be landed exactly as received, even if it's messy or malformed? *Answer:* Preserving the raw, unmodified data means nothing is lost if a downstream transformation turns out to be wrong — you can always reprocess from staging without re-extracting from the source system, which may not even retain the original historical state.

15. Describe how you'd handle a late-arriving dimension change — e.g., learning today that a customer's region actually changed two weeks ago. *Answer:* Determine the true effective date, find the dimension row that was current at that true date (not necessarily the row currently flagged as current), and split that row's date range at the true change date — shortening the old row's end date and inserting the new version with the correct start date — rather than naively expiring “as of today,” which would misattribute recent fact rows to the wrong dimension version.

End of Book 2. Continue to Book 3: “Medallion Architecture, Snowflake & dbt Implementation, and a Real-World Enterprise Data Warehouse Project.”